

Documentación Técnica Exhaustiva - EuroCoinCollector

1. Visión General del Proyecto

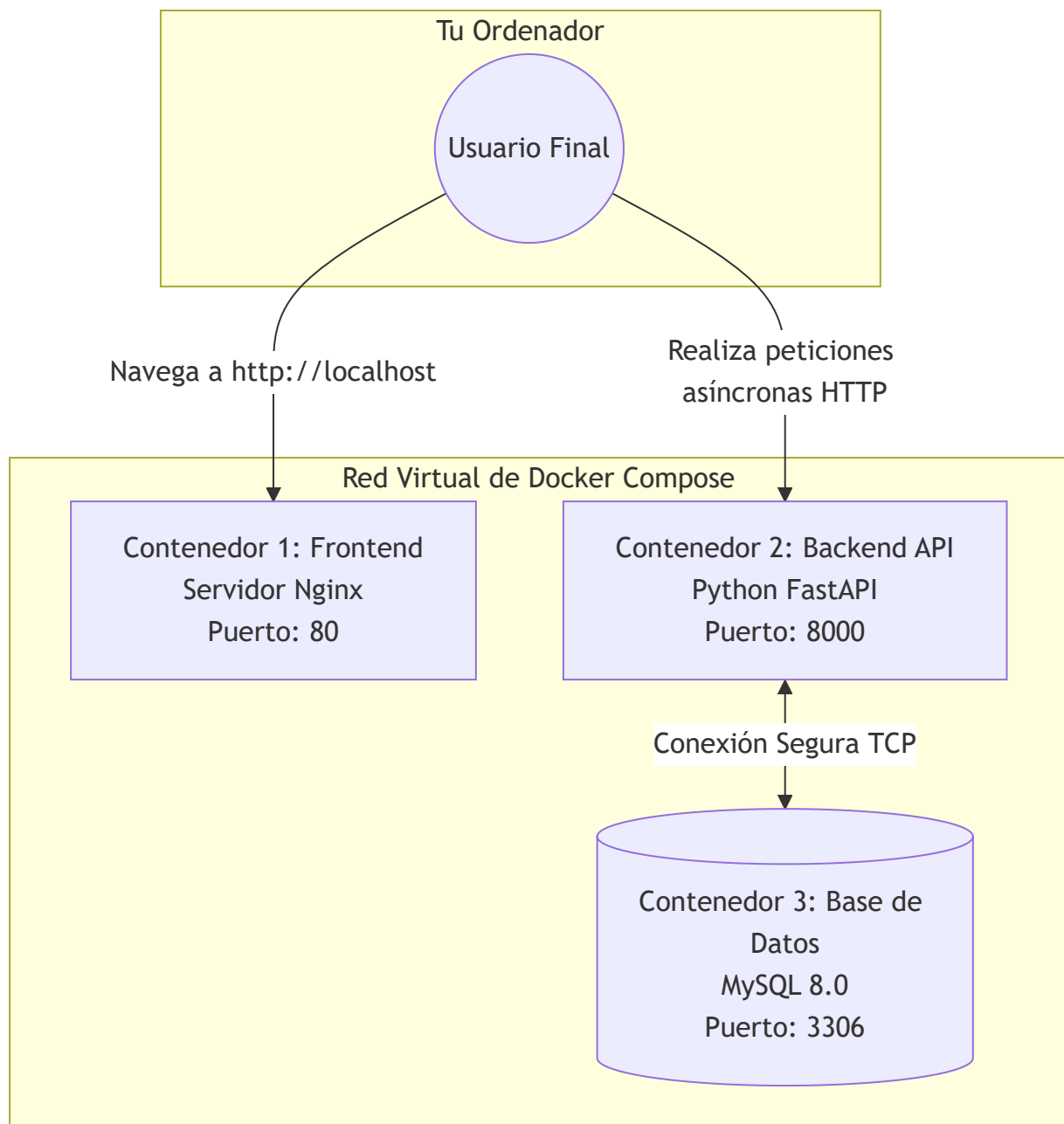
EuroCoinCollector es una plataforma integral Full-Stack diseñada para que los amantes de la numismática puedan gestionar su colección personal de monedas de 2 euros. La aplicación permite a los usuarios catalogar las monedas que poseen, buscar información histórica y valores de mercado, mantener una lista de deseos y gestionar ofertas de intercambio con otros coleccionistas.

Para garantizar que la aplicación sea robusta, escalable y muy fácil de desplegar en cualquier entorno sin problemas de compatibilidad, se ha desarrollado utilizando una arquitectura de microservicios.

2. Arquitectura de 3 Contenedores (Docker)

Docker es una tecnología de "encapsulación" (contenedorización). En lugar de instalar lenguajes de programación, librerías y bases de datos en tu ordenador directamente, Docker crea pequeñas "cajas fuertes" virtuales, llamadas contenedores. Cada contenedor tiene todo lo estrictamente necesario para hacer funcionar una sola pieza del rompecabezas.

Hemos dividido el proyecto en **3 contenedores independientes**:



1. El Contenedor Frontend (Nginx)

Contiene todo lo que el usuario ve y toca (HTML, CSS, JavaScript y las imágenes). Hemos utilizado Nginx, uno de los servidores web más rápidos y eficientes del mundo. Su única responsabilidad es recibir al usuario cuando entra a `http://localhost` y entregarle la página web al instante. Al estar separado, si el diseño web necesita actualizarse, el resto del sistema no se ve afectado.

2. El Contenedor Backend (FastAPI)

Es el "cerebro" de la operación. Construido con Python y el framework moderno **FastAPI**, no se encarga de dibujar pantallas ni botones, sino de procesar datos. Está programado para escuchar peticiones en el puerto 8000, validar información (usando Pydantic) y ejecutar lógica compleja antes de hablar con la base de datos.

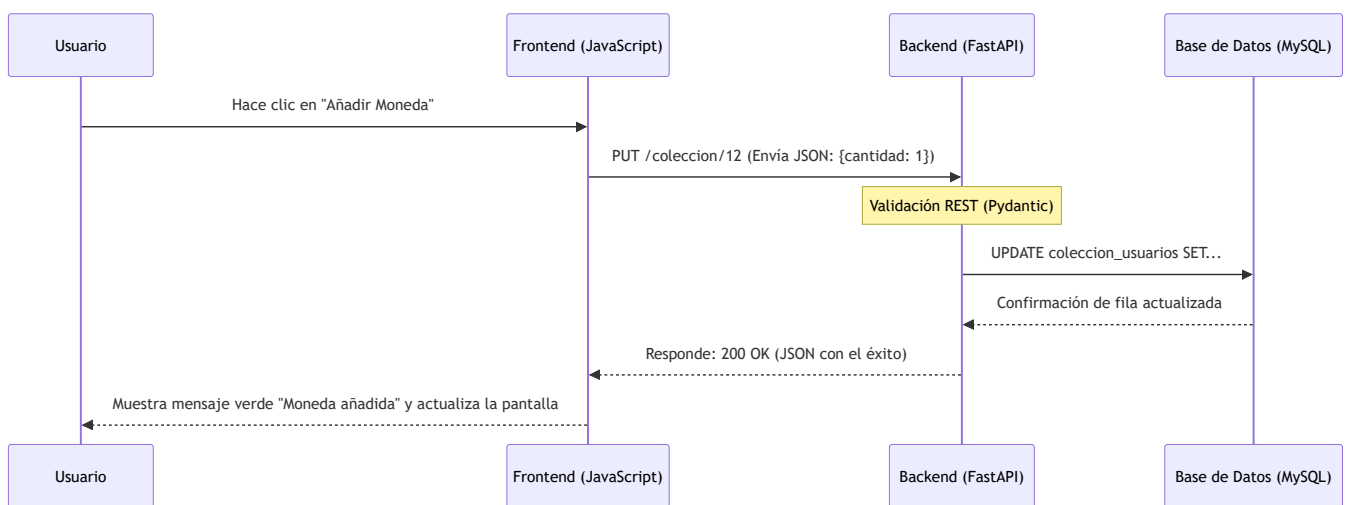
3. El Contenedor de Base de Datos (MySQL)

Es el "almacén". Utiliza el motor MySQL versión 8.0. Guarda de forma permanente todas las monedas, países, estadísticas de colección y ofertas. Como está encapsulado, si cambias de ordenador y te llevas el proyecto, la base de datos viaja contigo exactamente igual.

3. Conexión Frontend - Backend (Arquitectura REST)

La magia ocurre cuando estos contenedores se comunican. Utilizamos el estándar **REST (Representational State Transfer)**. REST es un conjunto de reglas que permite que el Frontend y el Backend hablen el mismo idioma (JSON) mediante métodos HTTP (`GET` , `POST` , `PUT` , `DELETE`).

Ejemplo paso a paso de una conexión: "Añadir a mi colección"



¿Por qué es tan potente esta separación? Porque gracias a REST, el Backend no necesita recargar la página web entera cada vez que haces clic. El JavaScript del Frontend usa `fetch()` por debajo para enviar los datos de forma silenciosa (asíncrona), y cuando FastAPI responde con un simple texto en formato JSON (`{ "status": "ok" }`), el Frontend actualiza solo ese trocito de la pantalla. Esto da la sensación de ser una aplicación móvil nativa ultrarrápida.

4. Orquestación con Docker Compose

Para que el profesor o un desarrollador no tenga que arrancar estos tres contenedores manualmente, usamos una herramienta llamada **Docker Compose**. A través del archivo `docker-compose.yml` , definimos cómo se construyen y cómo se conectan entre sí.

Con ejecutar un único comando mágico:

```
docker-compose up -d --build
```

Ocurre lo siguiente en segundos:

1. Docker crea una red virtual privada para que los 3 contenedores puedan verse.
2. Descarga la imagen base de MySQL, crea la base de datos eurocoleccion y la rellena con 50 monedas iniciales mediante un archivo init.sql.
3. Descarga una imagen pequeña de Python, instala FastAPI y las librerías necesarias.
4. Descarga Nginx y coloca nuestra web en su carpeta pública.
5. Los enciende todos en el orden correcto y los expone a tu navegador.

¡Cero configuraciones manuales, cero instalaciones pesadas en tu PC!
Todo encapsulado.